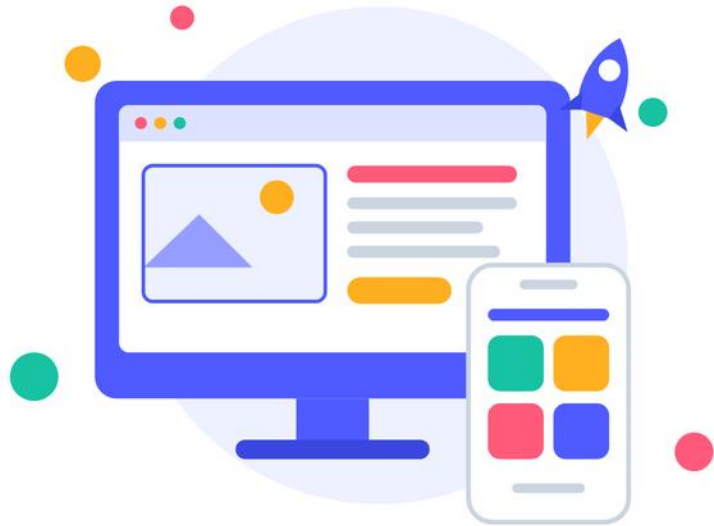


WEEK 05 · GRID · RESPONSIVE

THE PAGE GETS A SECOND DIMENSION

CSS Grid, and a page that fits every screen on purpose.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



WHERE WE ARE

Week 5 of 12. You can put two things side by side.
You cannot yet make a page.

THE NEXT 180 MINUTES

Five cycles, you typing in every one: the shell
Flexbox could not hold · a picture made of tracks · a
card row with no query · one prompt to Claude · the
shell again, from the phone up. Break at 90. The
assignment at 145.

WHAT THIS UNLOCKS

Every page you build from here. And next week
Tailwind just renames it.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180

the class assignment — resize the frame and watch it redraw

reload

open

Signal

SERVICES

Gateway

Checkout

Search

Mailer

Reports

Gateway

Everything on this page is one grid holding five regions, and one grid inside it holding a row of cards that nobody counted in advance.

REQUESTS

1.42M

Across all regions in the last twenty-four hours, including the retries that the gateway absorbed on the client's behalf.

updated 2 min ago

LATENCY

86ms

ERRORS

0.31%[Overview](#) [Activity](#) [Warm-ups](#)

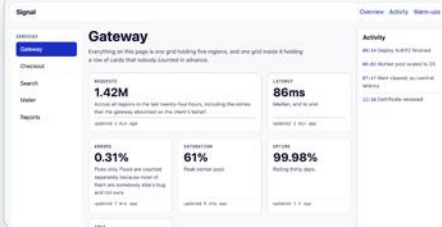
Activity

09:14 Deploy 4c81f2 finished

08:02 Worker pool scaled to 24

07:47 Alert cleared: eu-central latency

22:10 Certificate renewed



targets/week-05.png · 1280

[Signal](#) [Overview](#) [Activity](#) [Warm-ups](#)

Gateway

Everything on this page is one grid holding five regions, and one grid inside it holding a row of cards that nobody counted in advance.

375 · the same five regions, one column

► **THE POINT** — five regions, three arrangements, and each arrangement is a picture of names. The card row decides its own column count. There is not one `max-width` in the file. You type every piece of it today: once in a cycle, once in the assignment.

AI THIS WEEK

TUTOR. IT EXPLAINS; YOU TYPE.

LAYER · WEEKS 1-5 · PRIVATE TUTOR

ALLOWED

Ask it why `1fr` overflows, why `:nth-child(odd)` gives stripes.
Ask it to criticise CSS *you* typed.
Then check the answer in DevTools, at more than one width.

NOT ALLOWED

Pasting generated CSS into the assignment. **You type every character.** The skill this week: ask for a *rule of thumb* about your own grid, write it down as one sentence that could be false, then test it at the edges of your page — other widths, other conditions, a long word.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Cycle 4 is one prompt, twice, in whatever chat you have.

CYCLE 1 · 25 MIN

FLEX LOSES, GRID WINS: THE SHELL, ITS TRACKS AND ITS FLOOR

Last week's page in honest Flexbox, until a designer asks for a gap. Then the same five regions as a grid: one column that looks like nothing, a track list with commas that is thrown away in silence, percentages that forget the gap, and a URL that swells a column.

1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/01-flex-loses-grid-wins · a gap · display: grid · commas · 25% 50% 25% · 1fr 2fr 1fr · 1 / -1 · a long URL · minmax(0, ...)

2 WE DO

5 MIN

you type it on your copy · compare with checkpoint 1 · the grid badge draws four lines

3 YOU DO

8 MIN

warmup/ — six columns in a pattern of three, and one that goes all the way across

4 DEBRIEF

2 MIN

one column is not "Grid does not work" · 1fr is minmax(auto, 1fr) · reference

YOU DO · AI-FREE

[REFERENCE](#) · 4, 7

THIS IS WHAT YOU BUILT LAST WEEK

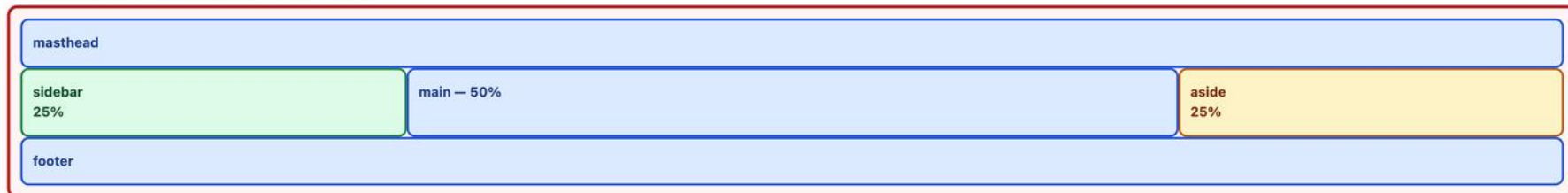
the page shell, attempted in honest Flexbox

↻ reload

open ↗

☐ add a gap between the regions ☐ shrink the percentages to compensate ☐ give the main region more content

FIVE REGIONS, ONE PAGE SHELL, FLEXBOX ONLY



The sidebar stops 48px above the footer.

```
container inner width 1498px
the band wants      1498px
gutter, sidebar to main 0px
sidebar bottom to footer 48px
wrapper elements needed 2
```

Nothing is wrong yet, and you should say so. Three percentages that add up to 100 do lay out three columns, and week 4 got you here honestly.

► **HANDS-ON** [demos/01-why-grid/01-flex-attempt/](#) — tick **ADD A GAP**, then **MORE CONTENT**. Do not defend it.

WHEN A TRACK LIST IS WRONG, NOTHING TURNS RED

A STRAY COMMA

```
1 .row {  
2   grid-template-columns: 1fr, 1fr, 1fr;  
3 }
```

Dropped. Falls back to `none`. You get one column.

THE FIX

```
1 .row {  
2   grid-template-columns: 1fr 1fr 1fr;  
3 }
```

Track lists are space-separated. Commas live inside `repeat()`.

► **THE POINT** — no console error, no red, no 404. It just looks like Grid did nothing.

TURN THE LINES ON

1 Elements panel: the container has a `grid` badge next to it.

2 Click it. The tracks are drawn on the page, numbered from the start
and
from the end.

3 Layout pane: *Show line numbers, Show track sizes, Show area names.*

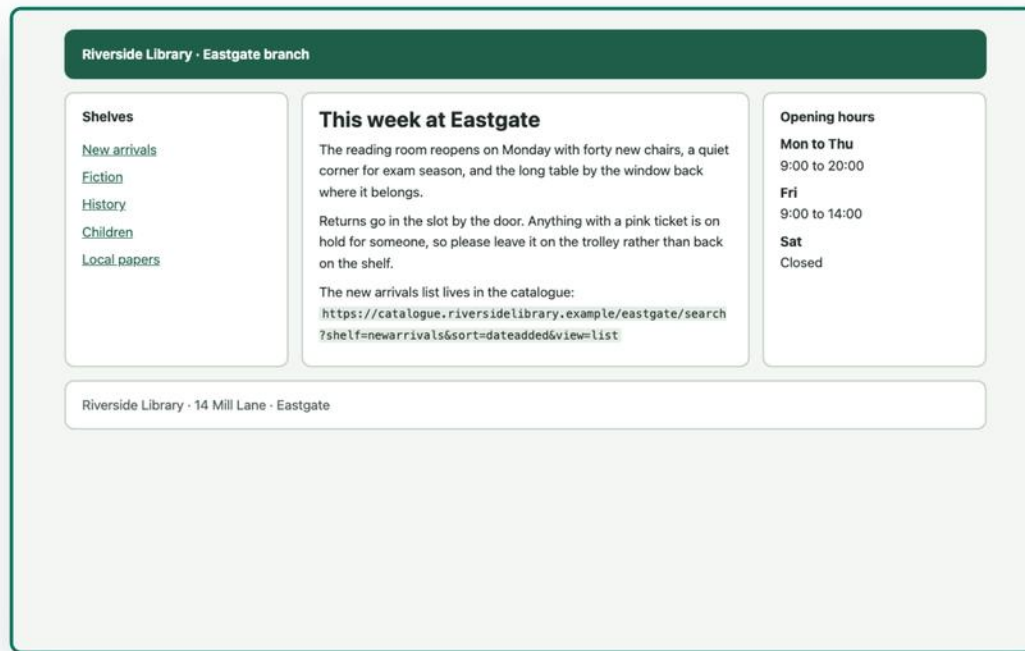
The badge is how you debug a grid: you stop guessing where line 3 is.

```
1 .tracks {  
2   grid-template-columns: repeat(2, 1fr 140px);  
3 }  
4  
5 .last {  
6   grid-column: 1 / -1;  
7 }
```

`repeat()` takes a **pattern**, not just one track. And `-1` is the last line, however many there are.

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/01-flex-loses-grid-wins/checkpoint.png` is this picture at 1280; `live-code/01-flex-loses-grid-wins/end/` is the code behind it. Five regions, the three in the middle ending on one line. At 768 nothing scrolls sideways, even with the URL. The grid badge draws four column lines.

1fr IS minmax(auto, 1fr)

BURSTS ON A LONG URL

```
1 .layout {  
2   grid-template-columns: 200px 1fr;  
3 }
```

THE FLOOR IS LOWERED

```
1 .layout {  
2   grid-template-columns: 200px minmax(0, 1fr);  
3 }
```

► **SAY IT OUT LOUD** — this course's own slide theme puts `min-width: 0` on every `.cols` child for exactly this reason.

CYCLE 2 · 25 MIN

A PICTURE MADE OF TRACKS

A poster from a sketch in pixels, and a tiled floor. The pixels become fractions, the square stops needing a height, every block is placed by line — and the obvious selector for the dark tiles draws stripes.

1 I DO

10 MIN

3 on the next two slides, 7 in the editor · live-code/02-picture-from-tracks · 360px 180px 60px · fr + height: 30rem · aspect-ratio: 1 · grid-area row / column / row / column · grid-column: span 2 · repeat(4, 1fr) · :nth-child(odd) · :nth-child(8n + 1)

2 WE DO

5 MIN

you type it on your copy · compare with checkpoint 2 · both still square at 375

3 YOU DO

8 MIN

the six-wide rug, and move the red dot · AI as tutor, on why, not on the answer

4 DEBRIEF

2 MIN

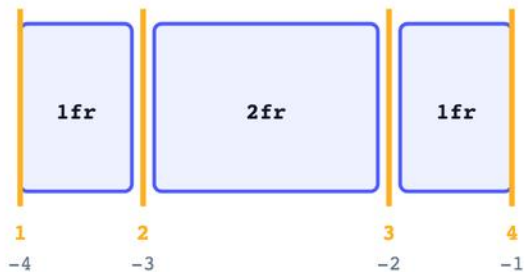
grid-area starts with the ROW · the period of a chequer · reference

YOU DO · AI AS TUTOR

➤ REFERENCE · 17

ITEMS SIT BETWEEN LINES, NOT IN CELLS

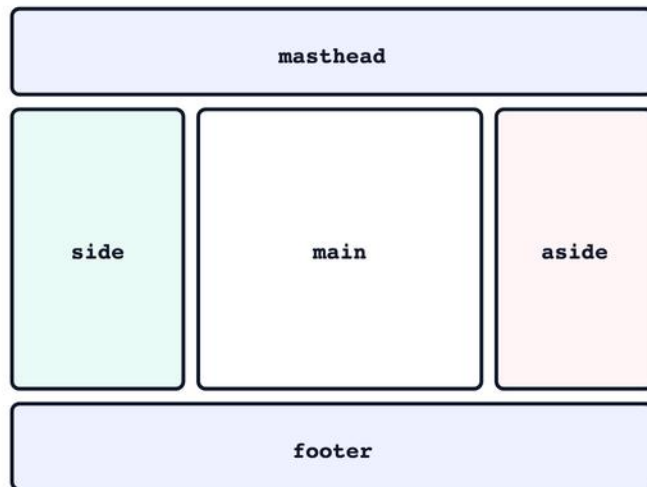
tracks, and the lines that come with them



an item sits between two lines



or draw the layout, and let the items claim a name



```
"masthead masthead masthead"  
"side    main    aside  "  
"footer  footer  footer  "
```

Three tracks. Four lines. The same line counted backwards — and `span 2`, which counts *tracks*, not lines.

A SQUARE WITH NO HEIGHT, AND A PATTERN WITH A PERIOD

NO HEIGHT ANYWHERE

```
.poster {  
  width: min(100%, 30rem);  
  aspect-ratio: 1;  
}  
  
.tiles > * {  
  aspect-ratio: 1;  
}
```

The width comes from the container, the height from the ratio. A fixed `height` is right at one width only — so a layout region never gets one.

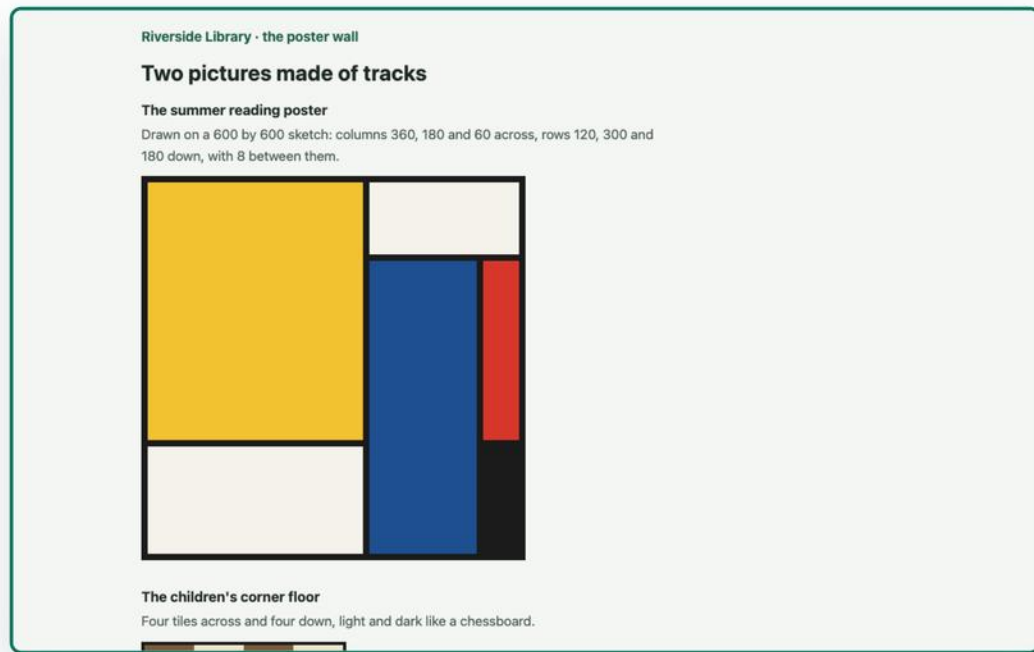
COUNT TO WHERE IT REPEATS

```
/* 4 across: odd tiles line up  
   in columns. Stripes. */  
.tiles > :nth-child(8n + 1),  
.tiles > :nth-child(8n + 3),  
.tiles > :nth-child(8n + 6),  
.tiles > :nth-child(8n + 8) { ... }
```

`an + b`: `a` is how many cells until the pattern comes back, `b` is where it starts. On an even-width board that is two rows.

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/02-picture-from-tracks/checkpoint.png` is this picture at 1280; `live-code/02-picture-from-tracks/end/` is the code behind it. At 375 the poster and every tile are still square, and nothing scrolls sideways. There is no `height` in the file.

CYCLE 3 · 30 MIN

A CARD ROW THAT NEEDS NO QUERY

Five events that will be seven next week, and three book covers in three shapes. The row decides its own column count, the covers share one shape without being squashed, and the inside of each card stays Flexbox.

1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/03-card-row-no-query · repeat(3, 1fr) at 375 · repeat(auto-fit, minmax(15rem, 1fr)) · auto-fill, then auto-fit · aspect-ratio: 2 / 3 · object-fit: cover · flex inside the card

2 WE DO

8 MIN

you type it on your copy · compare with checkpoint 3 · zero @media in the file

3 YOU DO

10 MIN

the staff row: no query, square photos that are not squashed, links on one line

4 DEBRIEF

2 MIN

aspect-ratio without object-fit · Grid for the row, Flex for the card · reference

YOU DO · AI-FREE

CYCLE 3 · THE ONE EVERYBODY GETS WRONG

A CARD ROW THAT NEEDS NO MEDIA QUERY

```
1 .cards {  
2   display: grid;  
3   grid-template-columns: repeat(auto-fit, minmax(15rem, 1fr));  
4   gap: 1rem;  
5 }
```

As many columns as fit, never narrower than 15rem, sharing what is left.

Four cards or nine, phone or monitor, and not one breakpoint.



AUTO-FIT AND AUTO-FILL, SIDE BY SIDE

demos/04-auto-fit-fill — two sliders, one difference ↶ reload open ↷

container width  700px items  8

```
repeat(auto-fill, minmax(140px, 1fr))
```



```
repeat(auto-fit, minmax(140px, 1fr))
```



Right now these two are identical, and that is the normal case.

container	700px
items	8

► **HANDS-ON** demos/04-auto-fit-fill/ — start at 8 items, where they are **IDENTICAL**. Then drag the count down.

IMAGES, IN THE ORDER THEY BITE

demos/09-images — press load, then the two toggles in order↻ reload↗ open ↗

load the images ☐ aspect-ratio: 3 / 2 ☐ object-fit: cover

A CARD ROW, AND THREE PICTURES THAT ARE NOT THE SAME SHAPE

3 : 2

2 : 3

1 : 1

row height now 67px

Nothing is loaded yet. Note the row height – then press the button and watch it change. That change is Cumulative Layout Shift, and it is the thing that moves a button out from under somebody's finger.

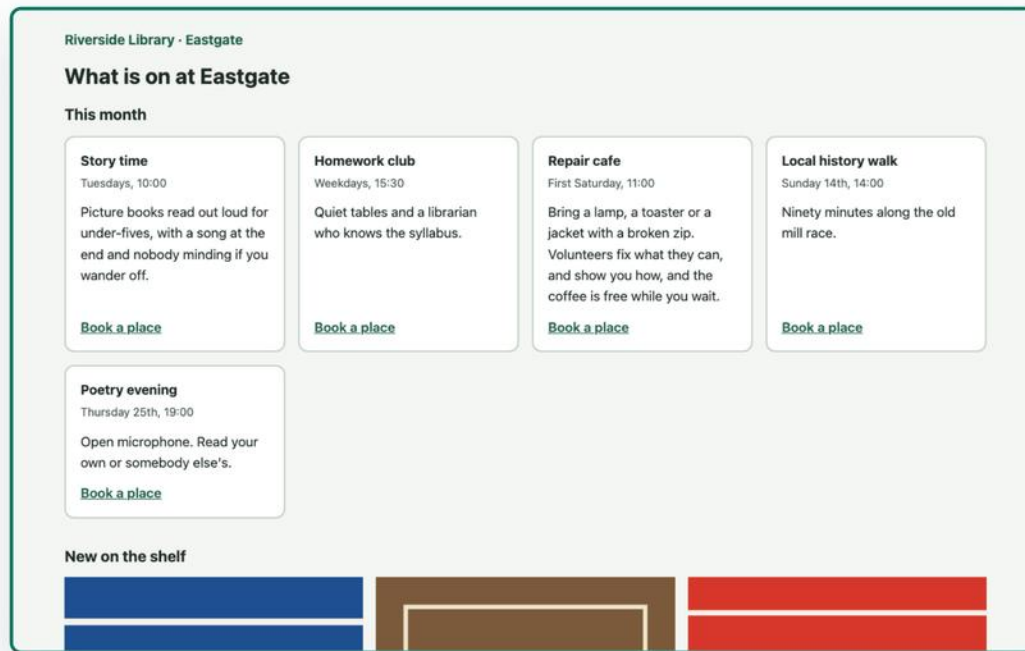
```
<!-- srcset and sizes: what the browser does BEFORE it downloads anything -->

```

► **HANDS-ON** `demos/09-images/` — read the layout shift number out loud. Then the code panel: `400w` is a file's width, and `sizes` is how wide the image is shown.

CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/03-card-row-no-query/checkpoint.png` is this picture at 1280; `live-code/03-card-row-no-query/end/` is the code behind it. At 375 everything is one column and nothing scrolls sideways. Every cover is 2 : 3 and not one is squashed. `@media` appears zero times in the file.

THE RULE, AND IT IS SHORT

GRID

The **page**. Anything with rows and columns that have to line up.

Shells, card grids, dashboards, galleries, forms with a label column.

FLEX

The **inside of a component**. One axis, and the sizes come from the content.

Navbars, button rows, a card's internals, toolbars, a chip with an icon.

NOT A COMPETITION

You don't have to choose one over the other — Flexbox can be used inside a grid cell.

BREAK

10 MINUTES

Back at 100. Cycle 4 is the AI cycle — have a chat window open, free or paid, Live Server running, and the device toolbar ready.

CYCLE 4 · 25 MIN

"GIVE ME A RULE OF THUMB"

A "what's on" page in the week-5 style: a card row, a shelf, a sidebar. You ask for a rule of thumb, not for code. The rule sounds right in general. Then you test it on your page.

1 I DO

7 MIN

2 on the next slide, 5 in the chat ·
ai-lab/04-rule-of-thumb · P1 live,
write the rule as one sentence that
could be false, then measure it at
700, 1100 and 1440 · transcript.md
if the room got something else

2 WE DO

6 MIN

everyone pastes P1 · write the rule
as a sentence · make the change it
suggests and read the numbers,
before the catch is named

3 YOU DO

10 MIN

write P2 with your own measured
numbers · hunt one more edge ·
the journal line, after the exemplar

4 DEBRIEF

2 MIN

a rule of thumb is true at one edge,
so write it as a sentence that can
fail · the journal exemplar

YOU DO · AI AS TUTOR

WHERE THE NUMBERS COME FROM

NOT THIS

```
/* phone */ 480px  
/* tablet */ 768px  
/* laptop */ 1024px
```

A table of devices from 2015. None of those phones exists any more.

THIS

Narrow the window until **your** content stops working. That width is the breakpoint, and it moved when the content changed.

Write it in `em`, so it moves with the reader's font size too.

► **SAY IT OUT LOUD** — you are designing for content, not for a catalogue of hardware.

CHECKPOINT 4

THREE QUESTIONS BEFORE THE NEXT CYCLE

1. Did you write the rule as **one sentence with a number or a width in it** — one a measurement could prove false — *before* you opened the browser?
2. Did you measure it on your own page at three widths or more, plus one more condition (fewer items, more items, or a long word), and read numbers instead of looking?
3. Does your journal line hold the rule, **a width and a measurement**, and a verdict that says where the rule is true — and not just "it worked / it didn't"?

Compare — `ai-lab/04-rule-of-thumb/checkpoint-he.md` has the same three questions; `artifact/start/` and `artifact/swap-auto-fit/` are the two pages, ready to open side by side. The shape of the journal line is `ai-exemplars/exemplars/prompt-journal-good.md`, last section.

CYCLE 5 · 20 MIN

DRAW IT, THEN BUILD IT FROM THE PHONE UP

Cycle 1's page, on a phone: three columns of seventy pixels. Rewritten as a picture of names, one column with no query, and two min-width queries that each redraw only the picture — and one row short a cell, thrown away in silence.

1 I DO

7 MIN

2 on the next slide, 5 in the editor ·
live-code/05-draw-then-mobile-first ·
grid-template-areas, one column ·
grid-area: name on every region ·
@media (min-width: 40em) ·
64em with a ragged row · the row completed

2 WE DO

5 MIN

you type it on your copy · compare with
checkpoint 5 · 375, 768 and 1280

3 YOU DO

6 MIN

polish: a fluid h1 with a rem term, a
hover that moves only transform ·
AI as tutor

4 DEBRIEF

2 MIN

count the cells in every row · min-width
composes, max-width fights ·
reference

YOU DO · AI AS TUTOR

➤ REFERENCE · 20

TWO RULES AREAS WILL NOT FORGIVE, AND WILL NOT MENTION

RAGGED ROWS

```
"head head head"  
"side main main aside"  
"foot foot foot"
```

Every row needs the same number of cells.

NOT A RECTANGLE

```
"side main main aside"  
"side main aside aside"
```

A named area must be a rectangle.
There is no L shape.

WHAT HAPPENS

The declaration is dropped, the property falls back to `none`, and you get plain auto-placement.

Nothing is logged.

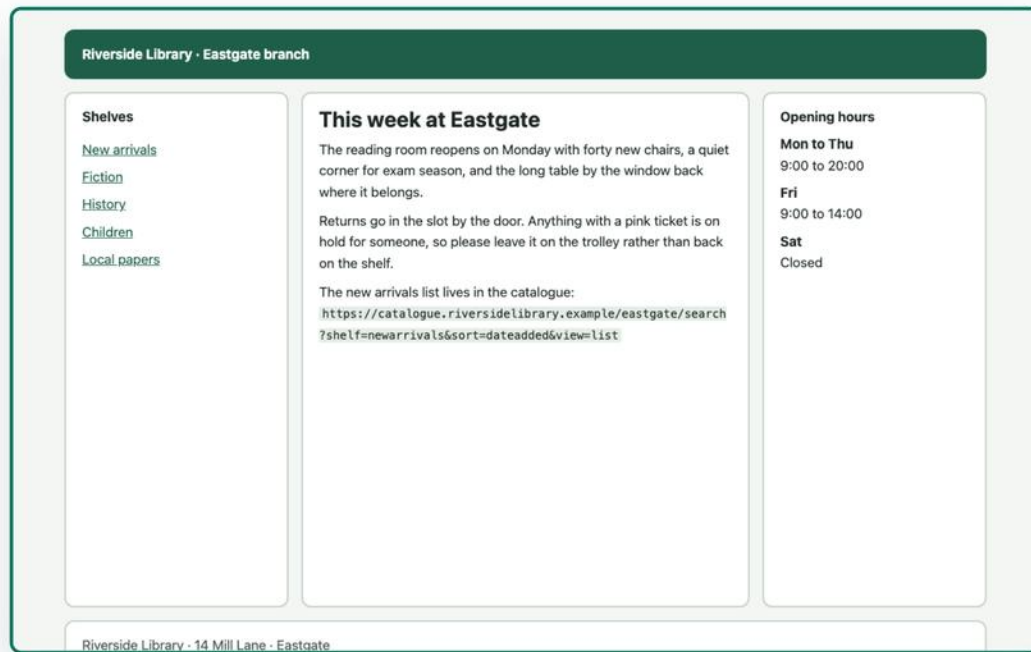
The rows, for a whole page:

```
.frame { min-height: 100dvh; grid-template-rows: auto 1fr auto; }
```

— the reading row is `1fr`. **dvh, not vh**: an empty page keeps the footer at the bottom.

CHECKPOINT 5

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/05-draw-then-mobile-first/checkpoint.png` is this picture at 1280; `live-code/05-draw-then-mobile-first/end/` is the code behind it. The columns are stretched to the window height by the `1fr` row; scroll and the footer is below. At 768: two columns and the hours below. At 375: one column, the reading region before the shelves. Every width query is `min-width`.

TYPE THAT DOES NOT STEP

demos/08-responsive/02-clamp [↶ reload](#) [open ↷](#)

viewport  480px ☒ keep the rem term in clamp() ☐ user has set a larger default font

font-size: 40px

Layout that fits the page

THREE STEPS, THREE MEDIA QUERIES

Layout that fits the page

CLAMP(24PX, 1REM + 2.5VW, 44PX)

Layout that fits the page

viewport 480px
root font-size 16px

clamp(24px, 1rem + 2.5vw, 44px)
minimum 24px
preferred 28.0px <- the PREFERRED value is winning

```
1 h1 {  
2   font-size: clamp(1.6rem, 1rem + 2.2vw, 2.6rem);  
3 }
```

POLISH, AND THE PROPERTY YOU MUST NOT ANIMATE

demos/10-motion – tick "busy" and run it again [reload](#) [open](#)

[run both](#) ☐ make the main thread busy

ANIMATING inset-inline-start AND width — LAYOUT EVERY FRAME

left

ANIMATING transform — COMPOSITOR ONLY

transform

```
frames drawn      0
average frame rate not measured yet
longest frame     — (16.7ms is one frame at 60fps)
frames over budget 0
```

ANIMATE

`transform` · `opacity` — the compositor, no layout

Colour on a short hover: no layout, but a repaint every frame. Fine, not free.

DO NOT

`width` · `height` · `top` · `left` · `margin` — and
`transition: all`

MIN-WIDTH COMPOSES. MAX-WIDTH FIGHTS.

demos/08-responsive/01-breakpoints – same output, two sources ↻ reload open ↗

viewport  380px

nav

A headline that has to fit
The width at which this stops fitting is the
breakpoint. Not 768.

aside

MOBILE-FIRST — MIN-WIDTH

```
.page {  
  grid-template-columns: 1fr;  
}  
@media (min-width: 560px) {  
  .page { grid-template-columns: 1fr 1fr; }  
  .nav { grid-column: 1 / -1; }  
}  
@media (min-width: 860px) {  
  .page { grid-template-columns: 200px 1fr 200px; }  
}
```

4 declarations, 1 block(s) applying now

DESKTOP-FIRST — MAX-WIDTH

```
.page {  
  grid-template-columns: 200px 1fr 200px;  
}  
@media (max-width: 859px) {  
  .page { grid-template-columns: 1fr 1fr; }  
  .nav { grid-column: 1 / -1; }  
}  
@media (max-width: 559px) {  
  .page { grid-template-columns: 1fr; }  
  .nav { grid-column: auto; }  
}
```

► **HANDS-ON** [demos/08-responsive/01-breakpoints/](#) — drag slowly and watch which blocks light up.

ASSIGNMENT · 145-175

THE ASSIGNMENT: ASSEMBLE WHAT YOU TYPED, THEN EXTEND IT

ASSEMBLE · CORE

`week-05/` is in your repo from `pre-class/`. The painting and the board are cycle 2, bigger. The shell is cycles 1 and 5: a picture of names, one column, two `min-width` queries. The card row is cycle 3 — and inside each `.kpi` it is still Flexbox.

EXTEND · STRETCH AND CHALLENGE

The status board is cycle 3's covers at `3 / 2`. The fluid number and the no-layout hover are cycle 5's "you do". Then the challenge: a wide card, the hole it leaves, and `subgrid` behind `@supports` — the next two slides, now; the challenge is finished at home.

THE RULES, AND THE EXAM

No `max-width` query. No fixed height on a layout region. Grid for the page, Flex for the component. AI as tutor — it explains, you type. SHA in Moodle. Home assignment 2 is out today. Week 13 asks about this code, offline.

THE GRID YOU DID NOT DECLARE

implicit grid Items past the end of your explicit tracks generate new rows. Sized by content unless you say otherwise.

grid-auto-rows The size of those rows. Important when items arrive from data.

grid-auto-flow **row** by default. **column** flips it.

dense Lets a later, smaller item climb back into a hole. Changes visual order, not the DOM — so tab order stops matching the screen.

```
1 .gallery {  
2   display: grid;  
3   grid-template-columns: repeat(3, 1fr);  
4   grid-auto-rows: 180px;  
5 }  
6  
7 .gallery {  
8   grid-auto-flow: dense;  
9 }
```

This is how a list of unknown length lays itself out.

SUBGRID, IN ONE SLIDE

A grid item that is itself a grid normally invents its own rows. With `subgrid` it takes them from the track it sits in — so the third line of every card in a row is the *same* line.

It is the honest answer to "make the footers line up when the cards hold different amounts of text".

```
1 .card {  
2   display: grid;  
3   grid-template-rows: subgrid;  
4   grid-row: span 4;  
5 }  
6  
7 @supports (grid-template-rows: subgrid) {  
8   /* wrap it, and keep the flex version  
9     as the fallback */  
10 }
```

DEBRIEF · 175–180

THREE QUESTIONS BEFORE YOU LEAVE

- 1 Three columns. How many lines, and what is the last one also called?
- 2 Six cards, room for four columns. What is different between `auto-fit` and `auto-fill` here? And with three cards?
- 3 Your sidebar refuses to shrink and the page scrolls sideways. One declaration.

► **GO DEEPER** — the reference deck is optional depth, and it is not examined: the Grid catalogue, every track-sizing keyword, placement and areas in full, why `1fr` has a floor, the period of a chequer, `aspect-ratio` against the width and height attributes, the Layout pane, container queries, and the demos behind today's cycles. The next slide lists it.

GO DEEPER

The reference deck for this week — every slide the lecture pointed at, in order.

[1 · The Grid catalogue](#) ← spine 33

[4 · You define the container first](#) ← spine 4

[5 · The Layout pane](#) ← spine 7

[6 · The tool you will use all evening](#) ← spine 6

[7 · Track sizing](#) ← spine 4

[10 · You have met this bug. It is the same bug.](#) ← spine 9

[11 · Why 1fr has a floor, and three ways to lower it](#) ← spine 9

[12 · repeat, auto-fit and auto-fill](#) ← spine 14

[17 · Placement](#) ← spine 10

[18 · A chequer's period, worked](#) ← spine 12

[20 · Named areas](#) ← spine 24

[22 · The implicit grid](#) ← spine 11

[24 · Alignment — the whole matrix](#) ← spine 19

[26 · Grid or Flex](#) ← spine 19

[27 · Now, the width you did not design for](#) ← spine 21

[28 · Responsive — the units](#) ← spine 21

[29 · clamp\(\), and the accessibility trap](#) ← spine 27

[30 · Images](#) ← spine 17

[31 · aspect-ratio and the width and height attributes](#) ← spine 12

[33 · Motion](#) ← spine 28

[35 · The five mistakes](#) ← spine 33

[36 · Where to look it up](#) ← spine 33